DrawingRobot · Kinematics & feedback linearisation at an off-axis output point

Lesson 01 — Pen-Path Tracking on a Differential-Drive Robot

May 1, 2026

Contents

Prerequisites

- HS baseline: trigonometry (sin, cos, rotations); vector arithmetic in 2-D; basic differential equations ("$\dot{x}$ means dx/dt").
- Math tools: 2×2 matrix inversion; the determinant criterion for invertibility; the chain rule.
- No prior robotics or control-theory background is assumed.

Learning goals

- State the kinematic equations of a differential-drive robot and explain why they are non-holonomic.
- Derive the pen-velocity Jacobian $J(\theta)$ for a pen at body-frame offset (p_x, p_y) and show that $\det J = p_x$.
- Explain why a single constant-input command cannot draw a corner of radius smaller than $|p_x|$.
- Write down the feedback-linearised tracking law (feedforward + P-feedback + J^{-1}) and explain what each term does.
- Compare the three drawing strategies the codebase offers (`goto`, `line_to`, `trace`) on a square and predict which one is appropriate for a given pen position.

---

1. Differential-drive kinematics

The chassis is a rigid rectangle of length L (along the heading axis) and width W (between the two wheels). Two independently driven wheels are mounted directly opposite each other on the two long sides. The midpoint of the line joining the two wheels — call it M — is the natural reference point: every body pose is described by the world coordinates (x, y) of M together with the heading θ (CCW from world $+x$).

The pen is bolted to the chassis somewhere on its outline, at a fixed body-frame offset (p_x, p_y) measured from M. By convention p_x is the offset along the heading direction and p_y is the offset perpendicular to it (toward the left wheel for $p_y > 0$).

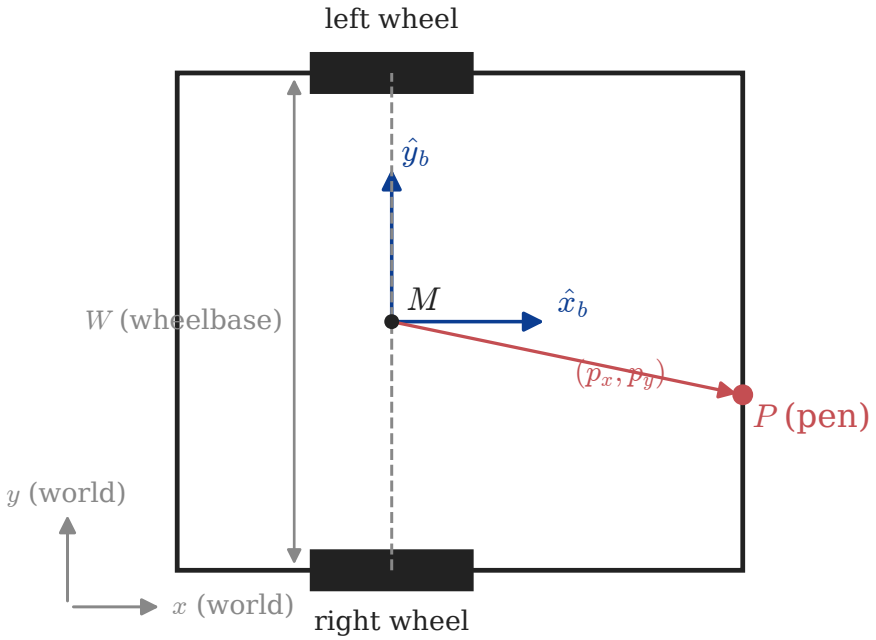

Figure 1: Chassis, wheels, body frame, and pen offset (p_x, p_y) measured from M.

Let v_l and v_r be the linear speeds of the left and right wheel contact points (positive = rolling forward). Two quantities summarise what the body is doing:

$$v = \tfrac{1}{2}(v_l + v_r), \qquad \omega = \frac{v_r - v_l}{W}$$

Here v is the body's linear speed along its current heading and ω is the body's angular speed (positive = CCW). These follow from "the wheel midpoint moves at the average wheel speed" and "the angular speed is the velocity difference divided by the lever arm W". Set $v_l = v_r$ to drive straight, $v_l = -v_r$ to spin in place, anything else to follow a circular arc.

In world coordinates the pose evolves as

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega.$$

These three equations are the entire kinematic model of M. Notice that $\dot{x}$ and $\dot{y}$ are coupled by θ: the body's velocity vector $(\dot{x}, \dot{y})$ is always along the heading direction $(\cos\theta, \sin\theta)$, never perpendicular to it. Eliminating v between the first two equations gives the **non-holonomic constraint**

$$\dot{x}\sin\theta - \dot{y}\cos\theta = 0,$$

or equivalently: M cannot move sideways. A car cannot slide; neither can M. The robot has two control inputs (v_l, v_r) but three pose variables (x, y, θ), so it is *under-actuated*.

For a constant-input command with $v_l \neq v_r$, M traces a circle whose centre — the **instantaneous centre of rotation (ICR)** — lies on the wheel-axis line at distance $R = v/\omega$ from M:

$$R = \frac{v}{\omega} = \frac{W}{2}\,\frac{v_l + v_r}{v_r - v_l}.$$

The pen, rigidly attached, traces a concentric circle about the same ICR (Figure 1.2).

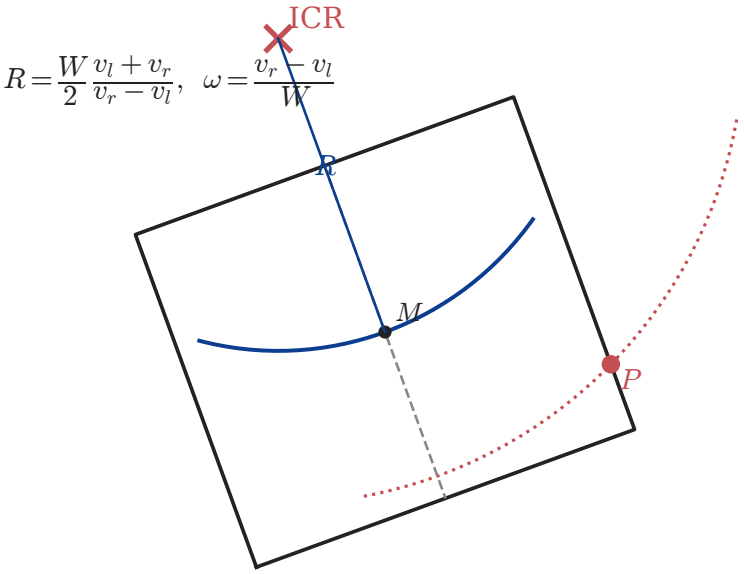

Figure 2: Constant (v_l, v_r) drives M along an arc about the ICR; the pen traces a concentric circle of radius $\|\text{ICR} - P\|$.

Exercise 1.1 *(simple — applies v, ω formulas)*

The simulator's wheelbase is $W = 20.4$ cm. The user issues `v_l = 8` cm/s, `v_r = 12` cm/s. (a) What is the body's linear speed v? (b) What is its

angular speed ω in rad/s? (c) Compute the ICR distance R. Is the body turning left or right?

---

2. The pen as an output point

The body is what we control; the pen is what we care about. Their relationship is fixed: at any instant,

$$P = M + R(\theta)\begin{pmatrix} p_x \\ p_y \end{pmatrix}, \qquad R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix},$$

so the pen's world coordinates are

$$P_x = x + p_x\cos\theta - p_y\sin\theta, \qquad P_y = y + p_x\sin\theta + p_y\cos\theta.$$

We want to know how P moves when the body moves. Differentiating P with respect to time and substituting $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$:

$$\begin{pmatrix} \dot{P}_x \\ \dot{P}_y \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -(p_x\sin\theta + p_y\cos\theta) \\ \sin\theta & p_x\cos\theta - p_y\sin\theta \end{pmatrix}}_{J(\theta)} \begin{pmatrix} v \\ \omega \end{pmatrix}.$$

$J(\theta)$ is the **pen Jacobian**. Each column has a clean meaning:

- Column 1 (v contribution): $(\cos\theta, \sin\theta)$ — pure forward translation moves the pen along the heading.
- Column 2 (ω contribution): rotating the body about M moves the pen perpendicular to $\overrightarrow{MP}$, with magnitude proportional to $\|MP\|$.

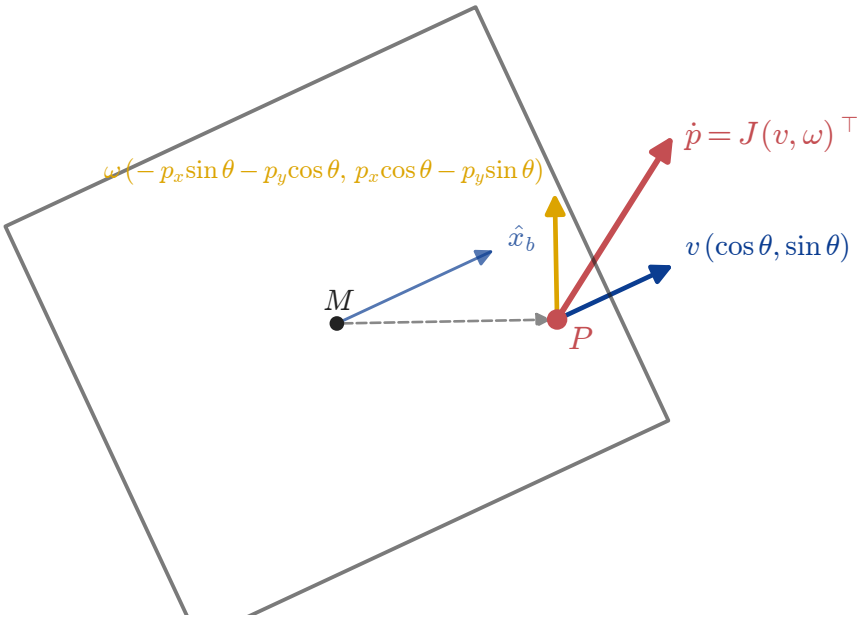

Figure 3: Pen velocity decomposed into a translation contribution (col. 1 of J) and a rotation contribution (col. 2 of J).

The geometric trick of the lesson is that this Jacobian is invertible *almost everywhere*. The criterion is its determinant.

> **Exercise 1.2** *(derivation — the conclusion is used as theory below)*
>
> Compute $\det J(\theta)$ using $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$. Show that the result simplifies, using $\sin^2\theta + \cos^2\theta = 1$, to a single number that depends only on the body-frame offset (p_x, p_y) and not on θ or p_y. State that number.

From Exercise 1.2, $\det J(\theta) = p_x$ for every θ — the determinant equals the pen's body-frame x-coordinate (its offset along the heading axis). The implications are stark:

- If $p_x \neq 0$, J is invertible at every heading. We can pick any commanded pen velocity $\dot{P}_{\text{cmd}}$ and solve for the body inputs that realise it: $(v,\omega)^{\top} = J(\theta)^{-1}\,\dot{P}_{\text{cmd}}$. The pen behaves like a fully actuated 2-D point.
- If $p_x = 0$, the pen sits on the wheel-axis line. J is singular for every θ. The pen inherits the body's non-holonomy and cannot move sideways from the heading direction.

The closed-form inverse, useful for the control law in §4: with $a = p_x\sin\theta + p_y\cos\theta$ and $b = p_x\cos\theta - p_y\sin\theta$,

$$J^{-1}(\theta) = \frac{1}{p_x}\begin{pmatrix} b & a \\ -\sin\theta & \cos\theta \end{pmatrix}.$$

So given a commanded $(\dot{P}_x, \dot{P}_y)$, the body inputs are

$$v = \frac{b\,\dot{P}_x + a\,\dot{P}_y}{p_x}, \qquad \omega = \frac{-\sin\theta\,\dot{P}_x + \cos\theta\,\dot{P}_y}{p_x}.$$

Both formulas blow up as $p_x \to 0$ — even *near* the wheel-axis line (small $|p_x|$), ω becomes large for a moderate commanded $\dot{P}$. That is something we will need to watch out for.

> **Exercise 1.3** *(simple — applies the result of 1.2)*
>
> A simulator user mounts the pen on the chassis outline at body-frame offset $(p_x, p_y) = (0, 10.2)$ cm — i.e., directly on top of the left wheel. Without computing anything, can the `trace` primitive (which depends on J^{-1}) work for this pen position? Justify in one sentence and state which alternative primitive should be used instead.

---

3. The single-arc lower bound on corner radius

Many drawing primitives — `arc`, `circle`, the rotation that precedes `forward` — are emitted as a *single* wheel command of constant (v_l, v_r) for some duration. The pen path painted by such a command is always a circular arc (or a line, or a point). What is the smallest circle the pen can paint with one such command?

In-place rotation is the easy case: $v_l = -v_r$, hence $v = 0$, and the body spins about M at $\omega = 2v_r/W$. The pen, at body offset (p_x, p_y), traces a circle about M of radius $\sqrt{p_x^2 + p_y^2} = |p_{\text{body}}|$.

For a general arc command with $\omega \neq 0$, the pen traces a circle about the ICR. The ICR sits on the wheel-axis line at distance $R = v/\omega$ from M — that is, at world position $M + R\,\hat{y}_b$ where $\hat{y}_b$ is the body y-axis. The pen's world position is $M + R(\theta)(p_x, p_y)^\top$. The radius of the pen's swept circle is the distance from ICR to P:

$$r_{\text{pen}}^2 = \|R\hat{y}_b - R(\theta)(p_x, p_y)^\top\|^2 = (R - p_y)^2 + p_x^2.$$

Choosing $R = p_y$ (i.e. placing the ICR directly on the line through P parallel to the heading) minimises this expression: $r_{\text{pen}}^2 = p_x^2$, so $r_{\text{pen}} = |p_x|$. Any other choice of R gives a strictly larger radius. So:

> **Single-arc lemma.** No constant-input command — including in-place rotation as the limit $R = 0$ — can paint a corner of radius smaller than $|p_x|$ on the page.

This is a **kinematic** floor, not an actuator one: even with infinitely fast wheels, a single command cannot do better. Geometrically, only points on the wheel-axis line have $|p_x| = 0$, so for a pen on the chassis outline, only the two wheels themselves can produce a sharp single-arc corner (Figure 1.5). A pen mounted anywhere else on the outline — including the front-mid or front-corner positions you actually want for visibility — is stuck with corner arcs of radius at least $|p_x|$ if you only ever issue one wheel command at a time.

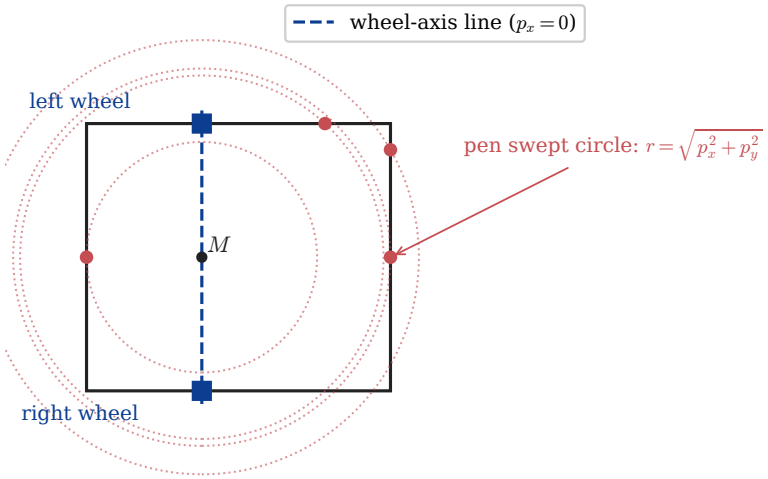

Figure 4: In-place rotation paints a pen circle of radius $\sqrt{p_x^2 + p_y^2}$. The wheel-axis line ($p_x = 0$, blue dashed) is the only sharp-corner locus, and the only outline points on it are the two wheel positions.

> **Exercise 1.4** *(simple — applies the lemma)*
>
> The simulator's default pen sits at body-frame offset $(p_x, p_y) = (14.4, 0)$ cm (front-mid mounting). (a) What is the smallest corner radius any single arc command can paint? (b) For a 30×30 cm square traced with this pen via `goto`, the rotation arcs at each corner have what radius? (c) Why does this make `goto` a poor choice for visibly square shapes with this pen?

---

4. Feedback-linearised tracking

The way around the single-arc bound is to **stop using a single command per move**. If we can update (v, ω) at every timestep, and we have $J^{-1}(\theta)$ to invert any commanded pen velocity into body inputs, then we can drive the pen along an arbitrary smooth path — and through arbitrary corners — by issuing a long sequence of small wheel commands.

The user's input is a polyline $V_0 \to V_1 \to \cdots \to V_n$ in pen coordinates, with V_0 the pen's current world position and $V_1, \ldots, V_n$ the user-supplied vertices. Each edge i has length $\ell_i = \|V_{i+1} - V_i\|$ and unit tangent $\hat{t}_i = (V_{i+1} - V_i)/\ell_i$. We pick a constant pen speed v_{pen} and a discretisation step Δt. At step k along edge i the desired pen position and velocity are

$$P_{\text{des}} = V_i + s\,\hat{t}_i, \qquad \dot{P}_{\text{des}} = v_{\text{pen}}\,\hat{t}_i, \qquad s = \min(k\,v_{\text{pen}}\,\Delta t,\ \ell_i).$$

A pure feedforward law — set $\dot{P}_{\text{cmd}} = \dot{P}_{\text{des}}$ each step — would track perfectly only if pose integration were exact. In practice, integration error drifts the pen off the desired path, so we add a proportional correction with gain K_p:

$$\dot{P}_{\text{cmd}} = \dot{P}_{\text{des}} + K_p\,(P_{\text{des}} - P_{\text{cur}}).$$

Then we apply the inverse Jacobian and the differential-drive map:

$$(v, \omega)^\top = J(\theta)^{-1}\,\dot{P}_{\text{cmd}}, \qquad v_l = v - \omega W/2, \qquad v_r = v + \omega W/2.$$

That is the whole loop. Block-diagram form:

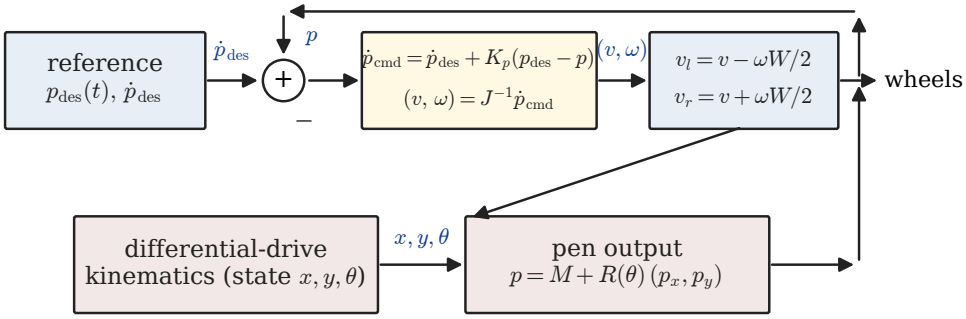

Figure 5: Per-timestep tracking loop: feedforward, P-feedback on pen position, J^{-1}, and the differential-drive map close the loop on P.

What does this look like at a sharp corner? At a vertex of the polyline, $\dot{P}_{\text{des}}$ flips direction in one timestep — from $v_{\text{pen}}\hat{t}_i$ to $v_{\text{pen}}\hat{t}_{i+1}$. The inverse-Jacobian solve still gives finite (v, ω), just with very large $|\omega|$ for one or two ticks: the body has to spin fast to swing the pen-tip around. The pen tracks the corner exactly within $O(v_{\text{pen}}\Delta t)$ rounding; the body trajectory underneath is generally not a polyline — it weaves and loops to make the pen-tip behave (Figure 1.4).

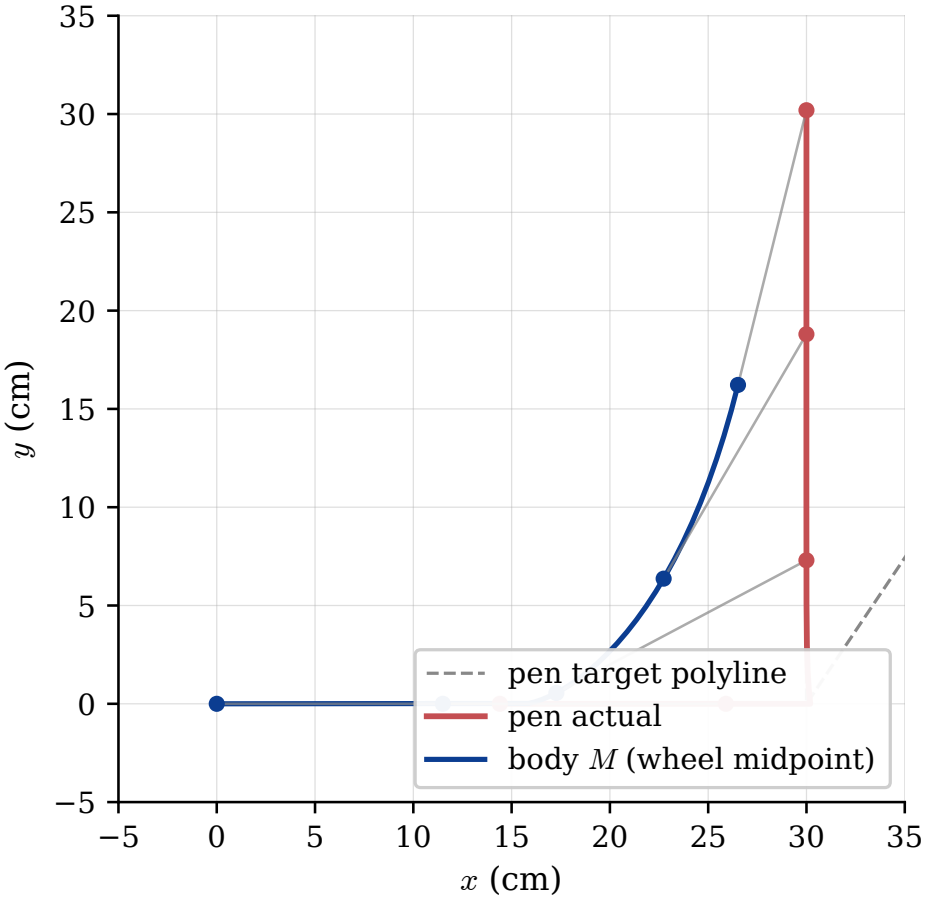

Figure 6: Pen tracks a sharp corner (red); the body M swings on a smooth curve underneath (blue). Grey segments connect simultaneous body/pen positions at five sample times. The body never visits the corner $(30, 0)$ — it cuts inside while the pen pivots through.

Three failure modes are worth knowing:

1. **Singularity at $p_x = 0$.** J^{-1} blows up. The codebase rejects `trace` for pens on the wheel-axis line; use `line_to` or `goto` there.
2. **Actuator saturation.** Sharp corners demand peak $|\omega|$ proportional to $1/p_x$ and to the corner sharpness. If wheels saturate at their max speed, the pen cuts the corner.
3. **Discretisation ringing.** $K_p\,\Delta t$ is the discrete-time feedback gain; if $K_p \gg 1/\Delta t$ the inner loop oscillates. The codebase's $K_p\,\Delta t = 8/120 \approx 0.067$ is comfortably stable.

The codebase's `trace` primitive (`drawingrobot/script.py:_plan_trace`) implements exactly this loop: $\Delta t = 1/120$ s, $K_p = 8\,\mathrm{s}^{-1}$, $v_{\text{pen}} = 12$ cm/s by default.

Exercise 1.5 *(simple — applies the inverse-Jacobian formula)*

The pen is at body offset $(p_x, p_y) = (14.4, 0)$, and the body heading is $\theta = 0$. The reference instructs $\dot{P}_{\text{des}} = (0, 10)$ cm/s — i.e. the pen should move purely

in the world $+y$ direction. Compute the body inputs (v, ω) that realise this, ignoring the P-feedback term. Comment briefly on why ω is nonzero even though the pen moves in a straight line.

---

5. Three drawing strategies, side by side

The codebase implements three pen-aware path commands. Each puts the unavoidable corner curvature in a different place:

- **`goto X Y`** — pen *lands* on (X, Y). Plans a single rotate-then-forward pair: the body first rotates in place to face the target, then drives straight. The rotation sweeps a pen circle of radius $|p_{\text{body}}|$ centred on M — so the corner curve is centred *between* polyline vertices and (for legs comparable to $|p_{\text{body}}|$) dominates the leg. Two wheel commands per leg.
- **`line_to X Y`** — pen *draws* a straight line from its current world position to (X, Y). Plans a "rotate-translate-rotate setup" that repositions M so the pen sits at the same world point but with the body now aligned to the new edge direction, then a forward leg. Each polyline edge is exactly straight on the page; the corner curvature is concentrated at the vertex (the setup translation has magnitude $|\Delta| = \sqrt{2}\,|p_{\text{body}}|$ for a 90° corner). Four wheel commands per leg.
- **`trace V_1 V_2 ... V_n`** — pen tracks the polyline via the feedback law of §4. Each edge is straight, each corner is sharp within $O(v_{\text{pen}}\Delta t)$. About $\ell_i/(v_{\text{pen}}\Delta t)$ wheel commands per edge — roughly 360 commands per 30 cm leg at the codebase defaults.

Figure 1.7 runs all three plans on the same 30 × 30 cm square with the same off-axis pen $(p_x, p_y) = (14.4, 0)$.

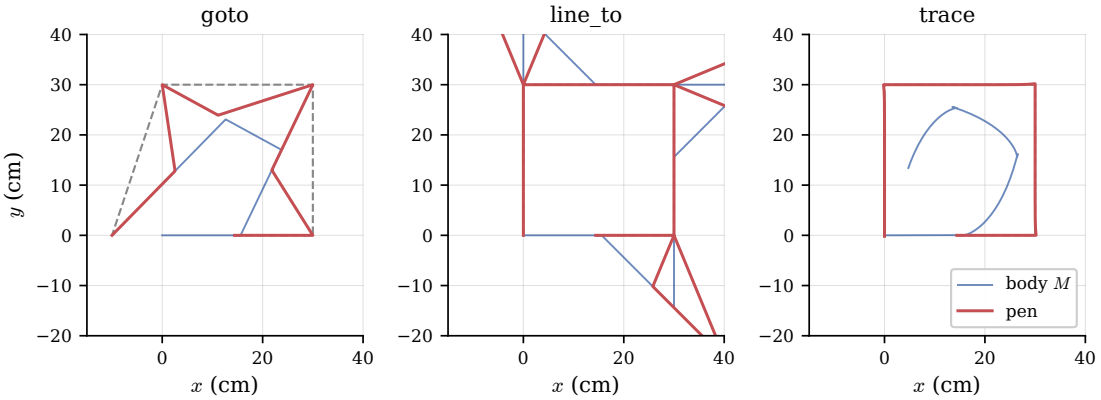

Figure 7: Pen and body trajectories on a 30 cm square, same pen offset for all three: `goto` (left), `line_to` (centre), `trace` (right). Body M in blue, pen in red, target polyline dashed grey.

The contrast tells the story. `goto` produces a chaotic shape because each leg's setup arc is centred between corners and is large on this scale. `line_to` paints the four edges

correctly, but each $\sqrt{2} \cdot 14.4 \approx 20.4$ cm setup translation projects a lobe outside the polyline at every vertex. `trace` produces a square — the pen lands on each vertex within ~ 0.2 cm of the target (dominated by integration drift over the full path) while the body M cuts inside, traversing small loops at each corner.

So: **with an off-axis pen, edges and corners are independent design choices.** `line_to` makes edges exact and corner curvature localised but visible. `trace` makes both edges and corners exact, at the cost of more wheel commands per second.

Exercise 1.6 *(simple — applies the strategy comparison)*

Suppose the pen is mounted at body offset $(p_x, p_y) = (0.5, 7)$ cm — close to but not on the wheel-axis line. (a) Is `trace` available? (b) The trace law would be available but undesirable here. State why, in one sentence, in terms of the inverse Jacobian.

End-of-chapter exercises

Exercise 1.E1 *(easy)*

For wheel speeds v_l, v_r on a chassis of width W, write down expressions for the body's linear speed, angular speed, and the radius of curvature of M. Check the limits $v_l = v_r$ and $v_l = -v_r$.

Exercise 1.E2 *(easy)*

A pen sits at body-frame offset $(p_x, p_y) = (10, -3)$ cm. The body is at world pose $(x, y, \theta) = (5, 2, \pi/4)$ cm/rad. Compute the pen's world position P.

Exercise 1.E3 *(medium)*

Show that the non-holonomic constraint $\dot{x}\sin\theta - \dot{y}\cos\theta = 0$ on the wheel midpoint M does **not** translate to a constraint of the form $\dot{P}_x f(\theta) + \dot{P}_y g(\theta) = 0$ on the pen position P, *provided* $p_x \neq 0$. (You may use the Jacobian J.) Interpret the result physically.

Exercise 1.E4 *(medium)*

Starting from $\det J = p_x$, show by direct calculation that the inverse Jacobian is

$$J^{-1}(\theta) = \frac{1}{p_x}\begin{pmatrix} b & a \\ -\sin\theta & \cos\theta \end{pmatrix}, \quad a = p_x\sin\theta + p_y\cos\theta, \quad b = p_x\cos\theta - p_y\sin\theta.$$

Use $J\,J^{-1} = I_2$ as the test.

Exercise 1.E5 *(hard)*

A user fixes the pen at $(p_x, p_y) = (14.4, 0)$ cm and wants the pen to traverse a 90° corner at constant pen speed $v_{\text{pen}} = 12$ cm/s — i.e. $\dot{P}_{\text{des}}$ flips from $(12, 0)$ to $(0, 12)$ in a single timestep $\Delta t = 1/120$ s. (a) What is the body-rotational impulse $|\Delta\theta|$ required to align the body during that one tick? (b) From this,

estimate the peak $|\omega|$ that the inverse-Jacobian law would demand at the corner. (c) The simulator's default ω_{rot} for in-place rotation is π rad/s. Will the wheels saturate at this corner?

Exercise 1.E6 *(hard — extension)*

The kinematic single-arc bound says no constant-input command can paint a pen circle smaller than $|p_x|$. Suppose we relax the "single command" restriction and allow exactly **two** consecutive constant-input commands. Show informally that the bound *cannot* be improved: any composition of two arc commands still has the property that the pen's worst-case curvature radius is at least $|p_x|$. (Hint: the body's pose at the join can be anything we like, but the second arc still has the same lower bound on its instantaneous curvature.) What does this say about why `trace` needs many commands per edge, not just a few?

Appendix A — Math used in this lesson

A.1 — Rotation matrices.

A rotation by angle θ in 2-D is

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}, \quad R(\theta)^{-1} = R(-\theta) = R(\theta)^{\top}, \quad \det R(\theta) = 1.$$

Used in §2 to express the pen's world position from its body offset, and in computing J^{-1}.

A.2 — Determinant and inverse of a 2×2 matrix.

For $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, $\det A = ad - bc$, and if $\det A \neq 0$,

$$A^{-1} = \frac{1}{\det A} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

The criterion $\det A \neq 0$ for invertibility is the central tool in §2 for deciding when the pen can be controlled freely.

A.3 — Time derivative of a rotated point.

If $P(t) = M(t) + R(\theta(t))\, p$ for a constant body-frame offset p, then

$$\dot{P} = \dot{M} + \dot{\theta}\, R'(\theta)\, p, \quad R'(\theta) = \begin{pmatrix} -\sin\theta & -\cos\theta \\ \cos\theta & -\sin\theta \end{pmatrix}.$$

This identity (chain rule on $R(\theta(t))$) is what produces the second column of $J(\theta)$ in §2.

A.4 — Pythagorean identity.

$\sin^2\theta + \cos^2\theta = 1$. Used in Exercise 1.2 to collapse the determinant computation to a single scalar p_x.

Appendix B — Symbols & units

Symbol	Meaning	Unit
x, y	World coordinates of the wheel-midpoint M	cm
θ	Body heading (CCW from world $+x$)	rad
W	Wheelbase (distance between the two wheels)	cm
L	Chassis length (along the heading axis)	cm
v_l, v_r	Linear speeds of left / right wheel contact points	cm/s
v	Body linear speed along its heading	cm/s
ω	Body angular speed (CCW positive)	rad/s
R	ICR distance from M to the centre of the body's arc	cm
p_x, p_y	Pen offset in the body frame (x along heading, y left)	cm
$P = (P_x, P_y)$	Pen position in world coordinates	cm
$J(\theta)$	Pen-velocity Jacobian, $\dot{P} = J(v, \omega)^\top$	dimensions of J are 1 (cols) and length (rows-cols mix)
V_i	Polyline vertex (target pen position)	cm
$\hat{t}_i$	Unit tangent of polyline edge i	(dimensionless)
v_{pen}	Commanded constant pen speed during `trace`	cm/s
K_p	Proportional gain on pen-position error	s^{-1}
Δt	Tracking timestep	s

Convention notes specific to this lesson: p_y is positive toward the left wheel (consistent with y-up in the body frame); body heading θ is measured CCW from world $+x$ in standard mathematics convention (not the pygame screen convention, where y points down — the simulator does the screen flip only at render time).

Appendix C — Source map

Every section of this lesson is grounded in the DrawingRobot source tree. The codebase is the primary reference; the project's `CLAUDE.md` summarises the architecture.

Section	Source
§1 — Differential-drive kinematics	`drawingrobot/kinematics.py` (the `step` function); `drawingrobot/robot.py` (`RobotGeometry`); `CLAUDE.md` "Robot model" section.
§2 — Pen as an output point, Jacobian, $\det J = p_x$	`drawingrobot/kinematics.py:transform_point`; the Jacobian and its inverse are derived in this lesson and implemented in `drawingrobot/script.py:_plan_trace`.
§3 — Single-arc lower bound	`CLAUDE.md` "Lower bound on corner radius" note; demonstrated in `scripts/square_pen.script` and `scripts/square_lines.script`.
§4 — Feedback-linearised tracking	`drawingrobot/script.py:_plan_trace` (the `trace` primitive); constants `TRACE_DT`, `TRACE_KP`, `DEFAULT_SPEED` in the same file.
§5 — Three drawing strategies	`drawingrobot/script.py:_plan_goto`, `_plan_line_to`, `_plan_trace`; `scripts/square_pen.script`, `scripts/square_lines.script`, `scripts/square_trace.script`.

Out-of-scope material in the codebase that is *not* used in this lesson: the rendering layer (`drawingrobot/sim.py`, `drawingrobot/ui.py`), the perimeter-parameter pen picker (`RobotGeometry.pen_offset`), and the geometry-changes-mid-run failure mode flagged in `CLAUDE.md` "Known issues" — that last item is a planning-vs-execution race condition, not a kinematic question, and belongs in a future lesson on the path/command layer.